PintOS 8팀 진행 계획

Oct 16, 2026 기준

1. 진행 방식

구현 범위를 **날짜별 단계**로 나누고, 각 단계에서는 **전원이 동일한 범위를 개별 구현**한다. 저녁 리뷰에서 하나의 버전으로 통합하며, 통합본이 다음 날 전원의 공통 출발점이 된다.

1.1 일일 사이클

시간	내용
오전 (1시간)	해당 단계 공동 설계. 정독 노트의 관련 섹션은 사전에 읽어 온다
오후	전원 동일 범위 개별 구현 (개인 브랜치)
저녁 마감	코드 리뷰 및 통합. 기준 코드를 선정하고 타 코드의 개선점을 반영한다
통합 이후	통합본을 `main`에 반영하고, 다음 날 전원이 이를 기준으로 새 브랜치를 생성한다

1.2 리뷰 규칙

- 리뷰어 모두 10~15분 정도 코드를 본다.
- 리뷰어는 질문을 최소 1개 이상 제기한다. (예: NULL 입력, 빈 문자열, 부모 선종료)
- 리뷰받는 사람은 리뷰어의 질문에 답한다.
- 기준 코드는 **당일 완료 기준 테스트 통과 여부**를 우선으로 선정하며, 동률일 경우 가독성을 기준으로 한다
- 미완료 인원의 코드는 개념 및 본인의 코드에 대해 설명한다.

1.3 브랜치 규칙

- 개인 브랜치명: `feat/단계/이니셜` (예: `feat/args/kmh`)
  - `args` → `syscall-basic` → `syscall-file` → `fork-wait` → `exec` → `rox-cleanup`
- `main`에는 리뷰를 거친 통합 코드만 반영한다
- 디버깅용 `printf`, `hex_dump`는 `main` 반영 전 반드시 제거한다. 출력 1줄만 남아도 테스트가 실패한다

1.4 주석 규칙

작성한 코드에는 반드시 주석을 단다. 리뷰에서 본인 코드를 설명하는 근거가 되며, 다음 날 통합본을 다른 인원이 이어받을 때 이해를 돕는다. 형식은 코드 컨벤션 4절을 따른다.

- 형식: `/* */` 만 사용하고, 새 주석은 한국어로 작성한다. 원래 영어 주석은 수정·삭제하지 않는다
- **새로 만든 함수**: 함수 위에 하는 일과 부르는 쪽이 지켜야 할 조건을 적는다 (예: 파일 시스템 락을 잡은 채로 불러야 한다)
- **`struct thread` 에 추가한 필드**: 필드 끝에 담는 값과 사용하는 기능을 적는다 (예: `/* exit 에 넘긴 종료 상태. exit · wait. */`)
- **기존 함수에 추가한 코드**: 추가한 블록 위에 왜 필요한지 적는다
- **동기화 지점**: 락·세마포어를 잡거나 내리는 곳마다 무엇을 기다리거나 보호하는지 적는다
- 무엇을 하는지보다 **왜 그렇게 하는지**를 적는다
  - 예: `/* 세마포어를 내린다. */` (X) → `/* 자식이 복제를 끝낼 때까지 부모가 fork 에서 리턴하면 안 되므로 기다린다. */` (O)
- 주석으로 막아 둔 옛 코드는 남기지 않는다
- 리뷰어는 주석이 없거나 코드와 맞지 않는 곳을 리뷰 댓글로 지적한다

2. Day 1 (10/16 금) — 인자 전달 착수

목표: 인자 전달을 구현하고 `hex_dump()` 로 유저 스택 배치를 검증한다.

비고: 이 시점에는 `write`, 종료 메시지, `process_wait` 이 미구현이므로 `args-*` 테스트는 통과하지 않는다. 당일 완료 여부는 테스트가 아닌 스택 출력으로 판단한다.

2.1 사전 학습 (약 1시간 30분)

1. 정독 노트 2번 섹션(Argument Passing)을 재확인한다
2. `args-single onearg` 기준으로 유저 스택 배치를 **손으로 직접 그린다.** 문자열 → 정렬 패딩 → `argv[2]` (NULL) → `argv[1]` → `argv[0]` → 가짜 리턴 주소 순서이며, `USER_STACK` 에서 시작하는 주소를 각각 계산해 둔다
3. `userprog/process.c` 의 `process_exec` → `load` → `setup_stack` 흐름을 읽고 각 함수의 역할을 주석으로 정리한다
4. `lib/string.c` 의 `strtok_r` 주석을 읽는다

2.2 단계별 구현 체크리스트

각 단계를 확인한 뒤 다음 단계로 넘어간다. 확인용 출력은 커널 `printf` 를 사용한다.

단계	구현 내용	확인 방법
1	커맨드라인을 공백 기준으로 단어 분리	분리된 단어와 개수를 출력. 공백 여러 개(`args-dbl-space` 형태)도 확인
2	첫 단어(프로그램 이름)로 실행 파일 열기	`load: … open failed` 메시지가 사라지는지 확인
3	문자열들을 스택 상단에 복사	각 문자열이 놓인 주소를 출력하여 사전 그림과 비교
4	스택 포인터를 8의 배수로 정렬	정렬 전후 주소 출력
5	NULL 및 각 문자열 주소를 역순으로 적재	`argv[0]`이 가장 낮은 주소인지 확인
6	가짜 리턴 주소 적재, `rdi`·`rsi`·`rsp` 설정	세 값 출력
7	전체 스택 덤프	`hex_dump()` 결과가 사전 그림과 일치하는지 확인

실행 명령 (`userprog/build`에서):

```
make
pintos --fs-disk=10 -p tests/userprog/args-single:args-single -- -q -f run
'args-single onearg'
```

2.3 완료 기준

- [ ] `hex_dump()` 결과가 argc 2, 문자열 2개, 정렬 패딩, NULL, `argv[1]`, `argv[0]`, 가짜 리턴 주소 순으로 일치한다
- [ ] `rdi = 2`, `rsi = argv[0]의 주소`이다
- [ ] 공백이 여러 개인 입력에서도 동일하게 동작한다
- [ ] 개인 브랜치에 커밋한다 (디버깅 출력은 다음 날 리뷰 전까지 유지 가능)

2.4 다음 날 리뷰 준비

각자 아래 질문에 답할 수 있도록 준비한다.

- 문자열을 먼저 넣고 포인터를 나중에 넣는 이유는?
- `argv[0]`을 마지막에 넣는 이유는?
- 정렬 패딩이 몇 바이트 생겼으며, 그 이유는?
- 원본 커맨드라인 문자열을 분리 과정에서 훼손해도 되는가?

3. Day 2 ~ Day 5 일정 (마감: 10/20 화)

3.1 전체 개요

일자	단계	완료 기준 테스트
Day 2 (10/17 토)	첫 유저 프로그램 실행 완료	`args-*`, `halt`, `exit`
Day 3 (10/18 일)	포인터 검증, 파일 시스템 콜	`create-*`, `open-*`, `close-*`, `read-*`, `write-*`, `bad-*`, `sc-*`
Day 4 (10/19 월)	fork, wait 완성, exec	`fork-*`, `wait-*`, `exec-*`
Day 5 (10/20 화)	실행 파일 쓰기 금지, 자원 정리, 전체 점검	`rox-*`, `multi-oom`, `filesys/base/*`, `make check` 전체

Day 4는 분량이 가장 많은 날이다. fork·wait가 당일 완료되지 않으면 exec는 Day 5 오전으로 이월하고, rox(소규모)와 병행한다.

당일 완료 기준 테스트를 통과하지 못하면 다음 단계로 넘어가지 않는다. 앞 단계 오류는 이후 모든 테스트 실패로 이어진다.

3.2 Day 2 — 첫 유저 프로그램 실행 완료

구분	내용
오전 리뷰	Day 1 인자 전달 리뷰 및 통합
오전 설계	`struct thread` 추가 필드 확정 (부모-자식 관계, 종료 상태, 대기용 세마포어). 부모·자식 중 선종료 시 자원 해제 주체 결정. **확정 즉시 1인이 `thread.h`에 커밋**하여 이후 충돌을 방지한다
사전 학습	정독 노트 4번 중 시스템 콜 진입 방식, `halt`, `exit`, `wait` / 5번 종료 메시지
개별 구현	시스템 콜 핸들러 골격(`rax` 기준 분기, 반환값 `rax` 기록), `halt`, `exit`, `write` (fd 1, `putbuf` 1회 호출), 종료 메시지, 설계 기반 `process_wait`
완료 기준	`args-single`, `args-multiple`, `args-many`, `args-none`, `args-dbl-space`, `halt`, `exit` 통과

Day 2 단계별 구현 체크리스트

분량은 단계 1~5가 비교적 적고, 단계 2·6의 `process_wait` 관련 작업이 가장 크다. `process_wait` 이 대기하지 않으면 유저 프로그램이 실행되기 전에 핀토스가 종료되어 이후 단계를 확인할 수 없으므로, **자식 대기(단계 2)를 먼저 구현**한다.

단계	구현 내용	확인 방법
1	`syscall_handler` 에서 `intr_frame` 의 `rax` 로 시스템 콜 번호를 읽고 번호별로 분기하는 골격 작성. 미구현 번호는 프로세스 종료	분기 진입 시 번호를 출력하여 어떤 시스템 콜이 호출되는지 확인
2	`process_wait` : 설계한 부모-자식 구조로 자식을 찾고, 자식이 종료될 때까지 대기. 자식은 종료 시 부모를 깨움	`Executing …` 이후 `system call!` 또는 단계 1의 출력이 보이고, 프로그램 실행 후에 핀토스가 종료되는지 확인
3	`halt` : `power_off()` 호출	`halt` 테스트 통과
4	`write` (fd 1): `rdi` · `rsi` · `rdx` 에서 fd·버퍼·크기를 꺼내 `putbuf()` 1회 호출, 반환값을 `rax` 에 기록	`args-single` 실행 시 `(args) begin` 등 유저 출력이 화면에 나오는지 확인
5	`exit` : 종료 상태를 설계한 필드에 저장한 뒤 스레드 종료	`exit` 시스템 콜 이후 프로세스가 정상 종료되는지 확인
6	종료 메시지: 유저 프로세스 종료 시 `이름: exit(상태)` 출력. 커널 스레드는 제외, 이름은 커맨드라인 전체가 아닌 프로그램 이름	출력 마지막 줄이 `args-single: exit(0)` 인지 확인
7	`process_wait` 완성: 자식의 종료 상태를 반환하고 자식 정보를 해제	`args-*` 결과 파일이 pass인지 확인

비고

- 포인터 검증은 Day 3 범위이므로, 이 단계의 `write` 는 유효한 버퍼만 들어온다고 가정한다. Day 3에서 검증을 추가한다
- 종료 메시지는 정상 `exit` 과 이후 커널이 강제 종료하는 경우 모두 한 번만 출력되어야 하므로, 출력 위치를 단계 6에서 정해 둔다
- 단계 1의 확인용 출력은 단계 7 확인 전에 제거한다. 남아 있으면 `.ck` 비교에서 실패한다

3.3 Day 3 — 포인터 검증, 파일 시스템 콜

구분	내용
오전 설계	유저 포인터 검증 방식 결정 (사전 검사 / page fault 활용). fd 테이블 구조와 크기. 파일 시스템 락 위치 (`load` 포함)
사전 학습	정독 노트 1번 Accessing User Memory, 3번, 4번 파일 관련 시스템 콜. `include/filesys/filesys.h` , `file.h` 함수 목록
개별 구현	포인터·버퍼·문자열 검증 함수, `page_fault` 에서 유저 프로세스 `exit(-1)` 처리, `create` `remove` `open` `close` `filesize` `read` (fd 0 포함) `write` (파일) `seek` `tell` , 파일 시스템 락
완료 기준	`create-*` , `open-*` , `close-*` , `read-*` , `write-*` , `bad-*` , `sc-*` 통과

Day 3 단계별 구현 체크리스트

검증 함수를 먼저 완성한 뒤 파일 시스템 콜에 적용한다. 이후 모든 시스템 콜이 이 검증을 거치므로, 검증이 틀리면 대부분의 테스트가 함께 실패한다.

단계	구현 내용	확인 방법
1	단일 유저 주소 검증: NULL, `KERN_BASE` 이상, 매핑되지 않은 주소일 경우 프로세스를 `exit(-1)` 로 종료	`*-null` , `*-bad-ptr` 계열 테스트에서 종료 메시지가 `exit(-1)` 로 출력되는지 확인
2	버퍼(시작 주소 + 크기)와 문자열(`\0` 까지) 검증: 시작뿐 아니라 끝까지 전 구간이 유효한지 확인. 페이지 경계에 걸치는 경우 포함	`*-boundary` , `sc-*` 계열 테스트
3	`page_fault` : 유저 코드에서 발생한 page fault는 프로세스를 `exit(-1)` 로 종료	`bad-read` , `bad-write` , `bad-jump` 계열 테스트
4	fd 테이블과 파일 시스템 락 준비: 프로세스 생성 시 초기화, fd 0·1은 콘솔용으로 비워 둠	다음 단계 테스트와 함께 확인
5	`create` , `remove`	`create-*` 테스트 (빈 이름, 긴 이름, 이미 존재하는 이름 포함)
6	`open` , `close` : 같은 파일을 두 번 열면 서로 다른 fd. 없는 fd·이미 닫은 fd 처리	`open-*` , `close-*` 테스트

단계	구현 내용	확인 방법
7	`filesize`, `read` (fd 0은 키보드), `write` (파일), `seek`, `tell`. 잘못된 fd, 크기 0, 콘솔 fd 방향이 반대인 경우 처리	`read-*`, `write-*` 테스트
8	회귀 확인	Day 2 완료 기준 테스트가 여전히 통과하는지 확인

비고

- 검증 실패로 종료할 때 파일 시스템 락을 잡은 상태라면 반드시 해제한다. 잡은 채로 종료하면 이후 다른 프로세스가 영원히 대기한다
- 파일 시스템 함수 호출은 모두 락 안에서 수행한다

3.4 Day 4 — fork, wait 완성, exec

구분	내용
오전 설계	fork 전체 흐름 (`process_fork` → `__do_fork`, 메모리 복제, fd 복제, 부모의 성공 여부 대기). wait 예외 처리 (직계 자식 여부, 중복 wait, 커널에 의한 종료 시 -1). exec 실패 시 처리와 fd 유지
사전 학습	정독 노트 4번 `fork`, `wait`, `exec`. 부록 Virtual Address. `threads/mmu.c`의 `pml4_for_each`
구현 방식	개별 구현이 어려울 경우 **공동 설계 후 1인 타이핑, 나머지 관찰**로 전환한다. 기능마다 타이핑 담당을 교대한다
우선순위	fork·wait를 먼저 완료한 뒤 exec에 착수한다
완료 기준	`fork-*`, `wait-*`, `exec-*` 통과

Day 4 단계별 구현 체크리스트

fork·wait(단계 1~6)를 먼저 완료하고, 통과 후 exec(단계 7)에 착수한다.

단계	구현 내용	확인 방법
1	부모-자식 연결: 자식 생성 시 설계한 구조로 부모가 자식을 기억하도록 등록	자식 생성 후 부모의 자식 목록을 출력하여 확인

단계	구현 내용	확인 방법
2	`fork` 시스템 콜에서 `process_fork` 호출: 부모의 현재 유저 레지스터 상태를 자식에게 전달할 방법 결정	자식 스레드가 생성되는지 확인
3	메모리 복제: `duplicate_pte`의 빈 부분 구현	`fork-once`에서 자식이 실행되는지 확인
4	fd 복제: 부모의 열린 파일을 자식도 같은 상태로 가짐	fd를 사용하는 `fork-*` 테스트
5	부모 대기와 반환값: 부모는 자식의 복제 성공 여부를 알 때까지 대기, 성공 시 자식 pid / 실패 시 `TID_ERROR`. 자식은 0 반환	`fork-*` 테스트 전체
6	wait 예외 처리: 직계 자식이 아닌 pid, 같은 자식 두 번 wait, 커널에 의해 종료된 자식은 -1	`wait-*` 테스트 전체
7	`exec` : 현재 프로세스를 새 프로그램으로 교체. 실패 시 `exit(-1)`, fd 유지, 스레드 이름 유지	`exec-*` 테스트
8	회귀 확인	Day 2, Day 3 완료 기준 테스트

비고

- 단계 5의 대기와 Day 2 `process_wait`의 대기는 목적이 다르다. 하나는 복제 완료, 하나는 종료를 기다린다
- exec의 커맨드라인 문자열은 유저 메모리에 있다. `process_exec`이 기존 주소 공간을 정리한 뒤에도 그 문자열을 쓸 수 있는지 확인한다
- 이 날은 개별 구현이 막히면 공동 설계 후 1인 타이핑으로 전환한다

3.5 Day 5 — 실행 파일 쓰기 금지, 자원 정리, 전체 점검

구분	내용
오전	실행 파일 보관 위치와 닫는 시점 설계 후 rox 구현 (실행 중 파일에 `file_deny_write`, 종료 시 해제). Day 4에서 이월된 exec가 있으면 함께 처리
오후	종료 시 해제 항목 점검: 열린 fd 전체, fd 테이블, 자식 정보, 실행 파일, 페이지 테이블. 이후 `make check` 실행, 실패 목록을 테스트 그룹별로 분담하여 원인 분석. 실패한 테스트의 `.c`와 `.ck`만 확인한다
사전 학습	정독 노트 5번 Denying Writes
완료 기준	`rox-*`, `multi-oom`, `filesys/base/*` 포함 전체 통과

Day 5 단계별 구현 체크리스트

단계	구현 내용	확인 방법
1	실행 파일 쓰기 금지: 실행 파일을 프로세스가 끝날 때까지 열어 두고 쓰기 금지. 현재 `load()`가 끝에서 파일을 닫는 동작과 충돌하지 않도록 처리	`rox-*` 테스트
2	실행 파일 해제: 프로세스 종료 시 실행 파일을 닫아 쓰기를 다시 허용	`rox-*` 테스트, 종료 후 같은 파일 쓰기 가능 여부
3	종료 경로 점검: 정상 `exit`, 커널 강제 종료, 로드 실패, fork 실패 각각에서 아래 해제 항목이 모두 처리되는지 확인	경로별로 코드 흐름 추적
4	해제 항목 점검: 열린 fd 전체, fd 테이블, 자식 정보 (부모가 먼저 종료한 경우 포함), 실행 파일, 보유 중인 락	`multi-oom`
5	동시성 점검: 여러 프로세스가 동시에 파일 시스템 콜을 호출하는 경우	`filesys/base`의 `syn-*`, `lg-*`, `sm-*`
6	전체 실행: `make check` 후 실패 목록을 테스트 그룹별로 분담	실패한 테스트의 `.c`, `.ck`만 확인
7	제출 정리: 디버깅 출력 제거, `main` 통합, `project2-final` 태그	`make check` 전체 통과

비고

- `multi-oom`은 해제 누락이 하나만 있어도 실패하며, 원인 위치가 출력에 나타나지 않는다. 단계 3·4의 체크를 먼저 끝낸 뒤 실행한다
- Day 4에서 exec가 이월된 경우 단계 1 전에 처리한다

4. 운영 규칙

4.1 막혔을 때

- 같은 문제로 **30분 이상 진전이 없으면 팀 채널에 공유**한다. 공유 시 실행 명령, 출력(패닉 메시지 포함), 의심 위치를 함께 적는다
- 패닉 발생 시 콜스택 주소를 `backtrace`로 변환해 첨부한다
- **마감 시각까지 미완료 시 완료된 인원의 코드를 기준으로 진행**한다. 미완료 인원은 리뷰에서 기준 코드 설명을 맡는다

4.2 테스트 실행

`userprog/build` 디렉토리 기준.

목적	명령
빌드	`make`
테스트 1개, 결과 파일 저장	`make tests/userprog/args-single.result`
테스트 1개, 화면 출력 확인	`pintos --fs-disk=10 -p tests/userprog/args-single:args-single -- -q -f run 'args-single onearg'`
전체 테스트	`make check`
결과 재실행	기존 결과 파일 삭제 후 재실행 (`rm -f tests/userprog/*.result`)

4.3 `main` 반영 전 점검

- [ ] 디버깅용 `printf`, `hex_dump`가 남아 있지 않다
- [ ] 새로 만든 함수·필드·추가 코드·동기화 지점에 "왜"가 담긴 한국어 주석이 있다 (1.4절)
- [ ] 당일 완료 기준 테스트를 통과한다
- [ ] 이전 단계 테스트가 여전히 통과한다
- [ ] `#ifdef VM` 블록 안에 코드를 추가하지 않았다
- [ ] `filesys` 디렉토리 코드를 수정하지 않았다

참고 문서: PintOS Project 2 명세 정독 노트, KAIST PintOS Project 2